B.Tech. BCSE497J - Project-I

REDUNDANCY-GATED RETRIEVAL: OPTIMIZED DIVERSIFICATION OF RAG CONTEXT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Programme

by

23BCT0093 PRATYUSH KUMAR V

23BCT0095 JEEVAANANDH V I

23BCT0097 NISHANT AJIT

Under the Supervision of

Dr. Thangaramya K

Associate Professor Grade 1

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

I hereby declare that the project entitled REDUNDANCY-GATED RETRIEVAL: OPTIMIZED DIVERSIFICATION OF RAG CONTEXT submitted by me, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by me under the supervision of Dr. Thangaramya K

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidate

CERTIFICATE

This is to certify that the project entitled REDUNDANCY-GATED RETRIEVAL: OPTIMIZED DIVERSIFICATION OF RAG CONTEXT submitted by PRATYUSH KUMAR V (23BCT0093), JEEVAANANDH V I (23BCT0095), NISHANT AJIT (23BCT0097), **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by him under my supervision during Fall Semester 2024-2025, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

Dr. J.Saira Banu

B.Tech CSE with Specialization in IoT

ACKNOWLEDGEMENTS

I am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired me to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for me. I appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

I express my profound appreciation to Dr. J.Saira Banu, the Head of the CSE Internet of Things, for her insightful guidance and continuous support. Her expertise and advice have been crucial in shaping the direction of my project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. Her constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project guide, Dr. Thangaramya K, for her dedicated mentorship and invaluable feedback. Her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. My supervisor's willingness to share her expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. Her support has not only contributed to the success of this project but has also enriched my overall academic experience.

Thank you all for your contributions and support.

Name of the Candidate

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**10**
1.	**INTRODUCTION**	**11**
	1.1 Background	**1**
	1.2 Motivations	**1**
	1.3 Scope of the Project	**1**
2.	**PROJECT DESCRIPTION AND GOALS**	
	2.1 Literature Review	
	2.2 Research Gap	
	2.3 Objectives	
	2.4 Problem Statement	
	2.5 Project Plan	
3.	**TECHNICAL SPECIFICATION**	
	3.1 Requirements	
	3.1.1 Functional	
	3.1.2 Non-Functional	
	3.2 Feasibility Study	
	3.2.1 Technical Feasibility	
	3.2.2 Economic Feasibility	
	3.2.2 Social Feasibility	
	3.3 System Specification	
	3.3.1 Hardware Specification	
	3.3.2 Software Specification	
4.	**DESIGN APPROACH AND DETAILS**	
	4.1 System Architecture	
	4.2 Design	
	4.2.1 Data Flow Diagram	
	4.2.2 Use Case Diagram	
	4.2.3 Class Diagram	
	4.2.4 Sequence Diagram	
5.	**METHODOLOGY AND TESTING**	

<< Module Description >>

<< Testing >>

6. PROJECT DEMONSTRATION

7. RESULT AND DISCUSSION (COST ANALYSIS as applicable)

8. CONCLUSION

9. REFERENCES

APPENDIX A – SAMPLE CODE

List of Figures

Figure No.	Title	Page No.
2.1	Project Gantt Chart	
4.1	System Architecture	
4.2	Data Flow Diagram	
4.3	Use Case Diagram	
4.4	Class Diagram	
4.5	Sequence Diagram	
7.1	Cost per query comparison	
7.2	Cost as k grows comparison	
7.3	ILS, Relevance comparison	

(In the chapters, figure caption should come below the figure and table caption should come above the table. Figure and table captions should be of font size 10.)

List of Tables

Table No.	Title	Page No.
5.1	Delta and Tau Values	
5.2	Metrics	
7.1	Metrics 1, 2 comparison	
7.2	MMR vs Gate Trip comparison	
7.3	Metrics 3, 4 comparison	

List of Abbreviations

GMC	Greedy with Marginal Contribution
GNE	Greedy Randomized with Neighborhood Expansion
MMR	Maximal Marginal Relevance
RAG	Retrieval Augmented Generation

Note: All the abbreviations should follow Alphabetical order

Symbols and Notations

δ	Duplicate threshold
λ	MMR relevance versus diversity weight
τ	Gate threshold
b	Typical similarity of two non-duplicate chunks
C(k,2)	Number of chunk pairs in a set of k chunks
c	Candidate chunk
d	Embedding dimension
E	Embedding matrix of corpus chunks
k	Number of chunks returned
max sim(c,s)	Highest similarity of c to any selected chunk
n	Number of chunks in the corpus
q	Query embedding
rel(q,c)	Relevance of chunk c to query q
s	Chunk already in the selected set
S	Chunk-to-chunk similarity matrix

ABSTRACT

Retrieval Augmented Generation (RAG) systems select the top k chunks most similar to a query and pass them to a language model. This plain top k retrieval often returns near duplicate chunks, so the model receives the same fact several times instead of k distinct pieces of evidence. Maximal Marginal Relevance (MMR) and later RAG specific extensions fix this by re-ranking the retrieved pool for diversity. These methods run their full diversification step on every query, even the many queries whose top k is already diverse and needs no correction.

This project proposes a redundancy gated retrieval method that adds a lightweight check before diversification. For each query, the plain top k is first computed as usual. A gate then measures the mean pairwise similarity within this set at a small fixed cost. If the set is already diverse, it is shipped without further work. If the gate signal is high, a repair step removes the most similar chunks and backfills the freed slots from a larger candidate pool, checking each replacement before it is added. This repair uses simple similarity thresholds rather than the continuous relevance versus diversity score used by MMR, so it is cheaper to run and easier to explain.

The method is built and evaluated on a small Wikipedia based corpus about diabetes, chosen because its three source articles share heavily overlapping content. Plain top k retrieval, full MMR, and the proposed gated method are compared on gate trigger rate, similarity comparisons per query, intra list similarity, relevance preservation, and duplicate rate. The central claim is that the gated method reaches diversity gains close to MMR while spending compute only on the queries that actually need it, since queries with an already diverse top k pay only the fixed cost of the gate check. This project targets a working end to end pipeline with all components implemented and evaluated on a representative query set.

Keywords - Retrieval Augmented Generation, Redundancy Gating, Query Result Diversification, Semantic Retrieval.

CHAPTER 1

1.1 INTRODUCTION

1.1 Background

Retrieval Augmented Generation combines a language model with a retrieval step that fetches relevant text chunks from a document collection before generating an answer. The usual approach ranks all chunks by similarity to the query and returns the top k highest scoring ones. This plain top k selection has a known weakness. Chunks that score highly for a query are often similar to each other, so the returned set can contain several near duplicate chunks instead of k different pieces of evidence. When this happens, the language model receives repeated information and effectively sees less context than the k slots suggest.

Maximal Marginal Relevance (MMR), introduced by Carbonell and Goldstein in 1998, was one of the earliest fixes to this problem. It re-ranks a candidate pool one chunk at a time, balancing relevance to the query against similarity to chunks already selected. Later RAG specific methods extend this same idea to chunk selection. All of these methods apply their diversification step to every query they process, regardless of whether the plain top k for that query was already diverse. This project studies whether that diversification step can be triggered only when it is actually needed, using a redundancy measurement computed directly from the retrieved chunks.

1.2 Motivations

Diversification methods such as MMR improve the quality of retrieved context, but they add computation to every single query. In a small deployment this cost may be negligible, but as query volume grows, always running a full re-ranking step becomes wasteful on the many queries whose plain top k already contains distinct, non overlapping chunks. Existing RAG literature has not questioned this always on assumption. Every diversification method surveyed for this project re ranks unconditionally, without first checking whether the ranking actually needs correction.

This project is motivated by that gap. If a lightweight signal can reliably tell whether a retrieved set is already diverse, then the more expensive repair step can be skipped entirely on those queries, and full computation is reserved for the queries where it is genuinely needed. This conditional approach does not aim to outperform MMR on diversity. It aims to match MMR level diversity on the queries that need it, while spending far less total compute across a full query set. For a student project with limited time and hardware, this framing also keeps the implementation small and explainable, since the core contribution is a simple gate and a repair step rather than a new ranking algorithm.

1.3 Scope of the Project

The scope of this project covers a working implementation of the redundancy gated retrieval method, along with the two baselines it is compared against, plain top k retrieval and full MMR. The pipeline is built in plain Python using sentence transformers for embeddings and NumPy for similarity computation, without frameworks such as LangChain or LlamaIndex, so every step remains visible and explainable to the team.

The corpus used for development and evaluation is a small set of three Wikipedia articles on diabetes, diabetes type 1, and diabetes type 2, chosen because their overlapping content produces genuine near duplicate chunks across documents. The project covers corpus preparation, embedding, the gate and repair logic, the MMR baseline, and an evaluation harness that measures gate trigger rate, compute cost, intra list similarity, relevance preservation, and duplicate rate across a query set.

The scope does not include training or fine tuning any embedding model, building a user facing application, or testing on multiple corpora. Dynamic, per dataset tuning of the gate and duplicate thresholds is also out of scope for this phase and is left as future work. The present goal is a correct, evaluated, end to end pipeline that demonstrates the core idea on one representative dataset.

CHAPTER 2

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Carbonell and Goldstein [1] introduced Maximal Marginal Relevance (MMR) to reduce redundancy in retrieval and summarization by combining relevance with novelty. Their greedy method picked, at each round, the candidate with the best combined score of query relevance and dissimilarity to items already chosen. MMR became the standard diversity method in information retrieval, but it needed manual tuning of its lambda parameter and re-ranked the full candidate pool on every query.

Vieira et al. [2] proposed a general framework to evaluate and optimize query result diversification and introduced two greedy methods, GMC and GNE, that gave more diverse results faster than earlier methods. Their framework always ran the diversification step, with no check for whether the retrieved set was already diverse.

Drosou and Pitoura [3] classified diversification definitions and algorithms into content based, novelty based, and coverage based categories, showing that no single definition of diversity fit every use case.

In later work, Drosou and Pitoura [4] extended diversification to streaming data using a cover tree structure that kept a diverse set updated incrementally, but the method still ran diversification continuously with no gate for already diverse states.

Zheng et al. [5] reviewed diversification techniques across databases, search, recommenders, and graphs, and noted that diversification had consistently been applied as an always-on step, flagging efficient query time diversification as an open problem.

Huang and Huang [6] surveyed the RAG pipeline across its retrieval and generation stages and identified redundant and irrelevant retrieved content as a recognized cause of poor RAG output quality.

Cuconasu et al. [7] studied how the type of retrieved passage affects RAG accuracy and found that irrelevant or redundant passages in the context can lower answer quality.

Salemi and Zamani [8] proposed a way to evaluate retriever quality specifically for RAG, arguing that a good RAG retriever should be judged by how much it helps the generator rather than by relevance alone.

Fan et al. [9] surveyed retrieval-augmented large language models and listed redundant or low-quality retrieved context as a persistent limitation of current RAG systems.

Huang et al. [10] surveyed causes of hallucination in large language models and noted that repetitive or irrelevant retrieved context is one factor that can lead a RAG based model to hallucinate.

Agarwal et al. [11] built a caching system for RAG pipelines and found that the same chunks are often retrieved again across different queries, motivating chunk level reuse to cut redundant computation.

Together, these works show diversification running always-on, from early frameworks [2] to MMR [1] to streaming extensions [3, 4], and RAG specific studies [6 to 11] confirming that redundant retrieved content remains an open problem for RAG systems. This project addressed that gap with a lightweight gate that triggers repair only when the retrieved top-k is measurably redundant.

2.2 Research Gap

Existing diversification methods, including MMR (Carbonell and Goldstein, 1998) and its RAG-era extensions, re-rank every query for diversity regardless of whether the plain top-k is already diverse. This always-on computation is spent even on queries that did not need it. No existing method conditionally triggers diversification based on a measured redundancy signal, so the computational cost cannot be reduced on the (typically majority) queries where the top-k is already acceptable.

2.3 Objectives

This project aims to achieve the following objectives within the current semester.

[1] Build the corpus and embedding pipeline, covering data cleaning, fixed size chunking, and a cached similarity matrix, completed as Build Stage 1 and 2.

[2] Implement plain top k retrieval and a faithful full MMR baseline, instrumented to count similarity comparisons, completed as Build Stage 3 and 4.

[3] Implement the redundancy gate and the dedup plus backfill repair step, with all shape deciding rules documented, completed as Build Stage 5 and 6.

[4] Build an evaluation harness that computes gate trigger rate, compute cost, intra list similarity, relevance preservation, and duplicate rate for all three methods on a shared query set, completed as Build Stage 7.

[5] Run the full pipeline across the complete query set, collect results, and produce comparison tables including a sensitivity sweep over the gate threshold, completed as Build Stage 8 and 9.

[6] Complete the project writeup and presentation material summarizing the method, results, and honest limitations, completed as Build Stage 10.

Each objective is measurable against the corresponding build stage checklist and is scheduled within the project timeline in section 2.5.

2.4 Problem Statement

Standard top k retrieval in RAG systems frequently returns near duplicate chunks, reducing the amount of distinct evidence available to the language model. Existing fixes such as MMR correct this by re-ranking every query, which adds computation even to queries whose top k is already diverse. There is no established method that first checks whether a retrieved set actually needs diversification before spending compute on the correction. This project addresses that specific gap by designing and evaluating a redundancy gate that triggers a cheaper repair step only when the retrieved set is measurably redundant.

2.5 Project Plan

The project plan follows the ten build stages defined for this project, from corpus preparation through evaluation and writeup. A Gantt chart mapping these stages to calendar dates is pending final scheduling by the team.

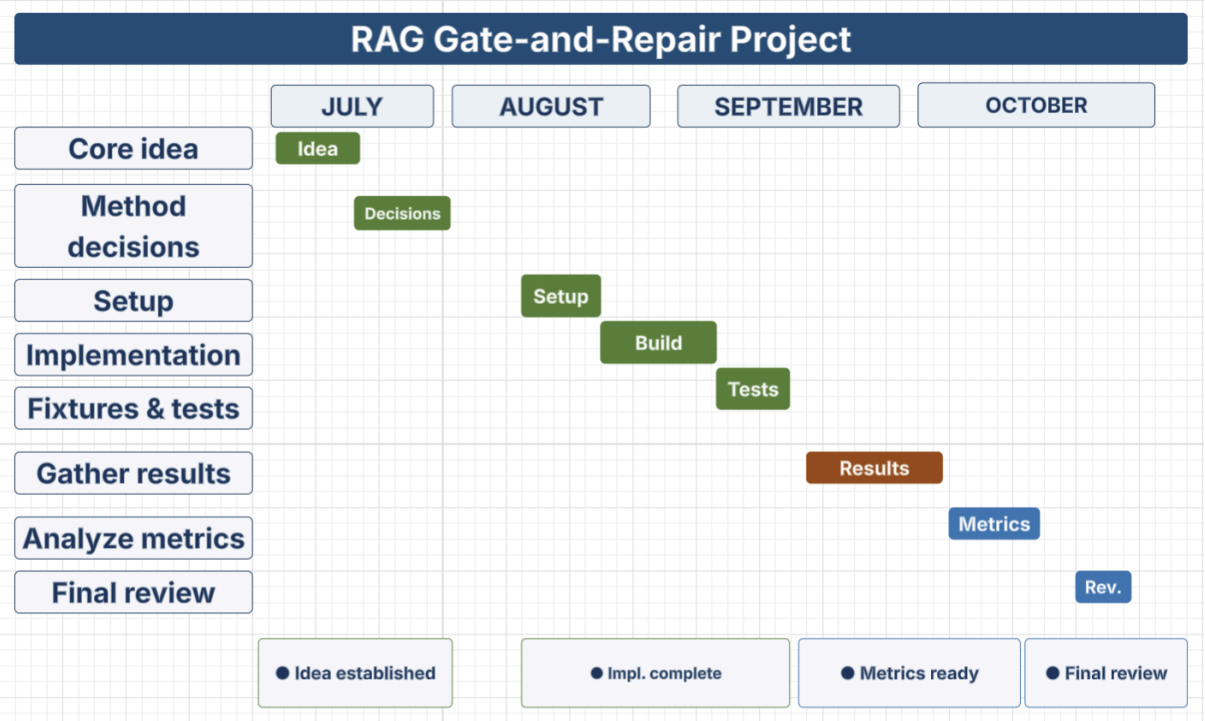


Fig. 2.1. Project Gantt Chart

CHAPTER 3

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

- The system shall load and clean the fixed Wikipedia corpus of three diabetes related articles.
- The system shall split cleaned articles into fixed size, non overlapping chunks.
- The system shall generate normalized embeddings for all chunks and queries using the BGE base model.
- The system shall compute and cache a full chunk to chunk similarity matrix.
- The system shall retrieve a candidate pool and the plain top k chunks for each query.
- The system shall compute a redundancy gate signal on the plain top k and decide whether to trigger repair.
- The system shall perform dedup and backfill repair when the gate trips, and return exactly k chunks.
- The system shall implement a full MMR baseline for comparison.
- The system shall compute gate trigger rate, compute cost, intra list similarity, relevance preservation, and duplicate rate for all three methods on a shared query set.

3.1.2 Non-Functional

- Performance: the gate check shall add only a fixed, small number of similarity comparisons per query regardless of whether it trips.
- Explainability: every component shall be implemented in plain Python without opaque frameworks, so all three team members can read and modify the code.
- Reproducibility: chunk ordering, tie breaking, and thresholds shall be fixed in configuration so results do not vary between runs.
- Maintainability: undecided design choices shall be flagged explicitly in configuration and code rather than resolved silently.
- Portability: the pipeline shall run on a standard laptop without requiring a GPU.

3.2 Feasibility Study

3.2.1 Technical Feasibility

- The full stack (sentence-transformers, NumPy, PyYAML) is open source, well documented, and already used successfully in similar retrieval projects.

- The corpus is small (136 chunks), so computing and caching the full similarity matrix is inexpensive and needs no specialized hardware.
- The team has working familiarity with Python, and the pipeline avoids opaque frameworks so each stage stays inspectable.

3.2.2 Economic Feasibility

- All software used is free and open source, so the project carries no licensing or infrastructure cost.
- The project runs on personal laptops with at least four cores and hyperthreading, requiring no cloud compute budget.

3.2.3 Social Feasibility

- A method that skips unnecessary computation makes diversified retrieval more accessible on ordinary hardware, which touches on Sustainable Development Goal 9 (Industry, Innovation and Infrastructure).
- Reducing redundant chunks in the retrieved context means end users get more complete answers without an increase in token cost.

3.3 System Specification

3.3.1 Hardware Specification

- Processor: minimum 4 CPU cores with hyperthreading
- Memory (RAM): standard laptop configuration, no GPU required
- Storage: sufficient for corpus files, cached embeddings, and the similarity matrix
- Machine: agnostic, any personal computer meeting the above specification

3.3.2 Software Specification

- Operating System: any (Windows, Linux, or macOS)
- Programming Language: Python 3.13
- Development Environment: Visual Studio Code, Neovim
- Libraries and Frameworks: sentence-transformers, NumPy, PyYAML
- Version Control: Git
- Embedding Model: BAAI/bge-base-en-v1.5

CHAPTER 4

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

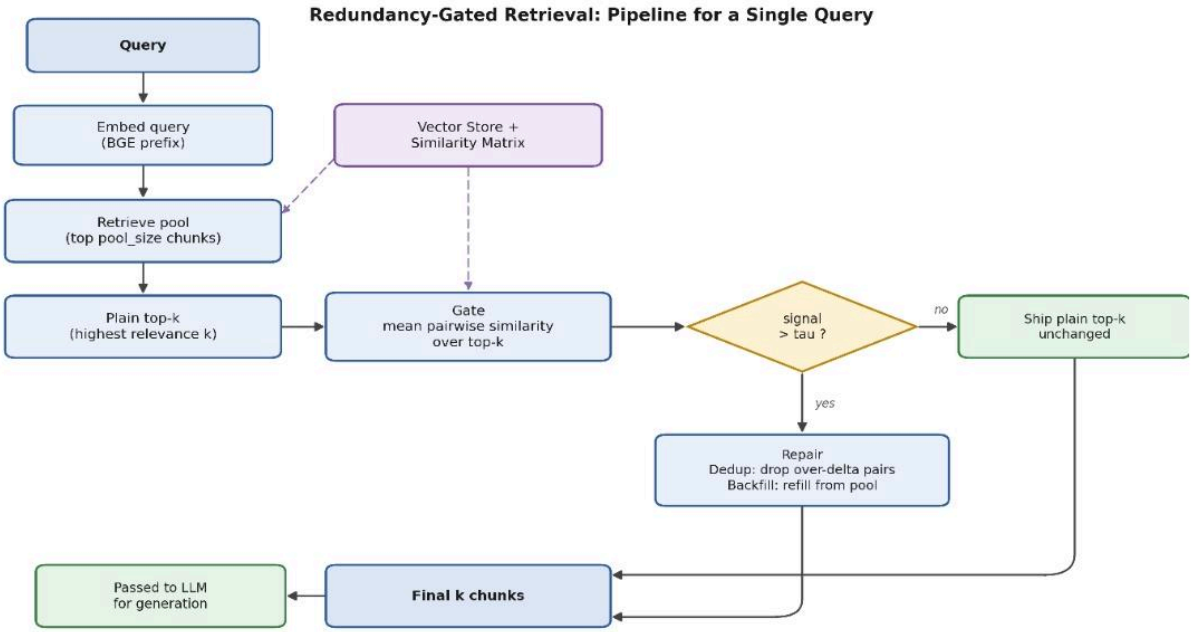


Fig. 4.1. System Architecture

Figure 2 shows the pipeline for a single query. The query is embedded and compared against the cached corpus similarity matrix to retrieve a candidate pool. The plain top k chunks from this pool are passed to the gate, which computes the mean pairwise similarity among them at a small fixed cost. If the signal is at or below the threshold, the plain top k is shipped without further work. If the signal is above the threshold, the repair step removes the most similar chunks and backfills the freed slots from the remaining pool, checking each candidate before it is added. The final k chunks are then passed to the language model for generation.

4.2 Design

4.2.1 Data Flow Diagram

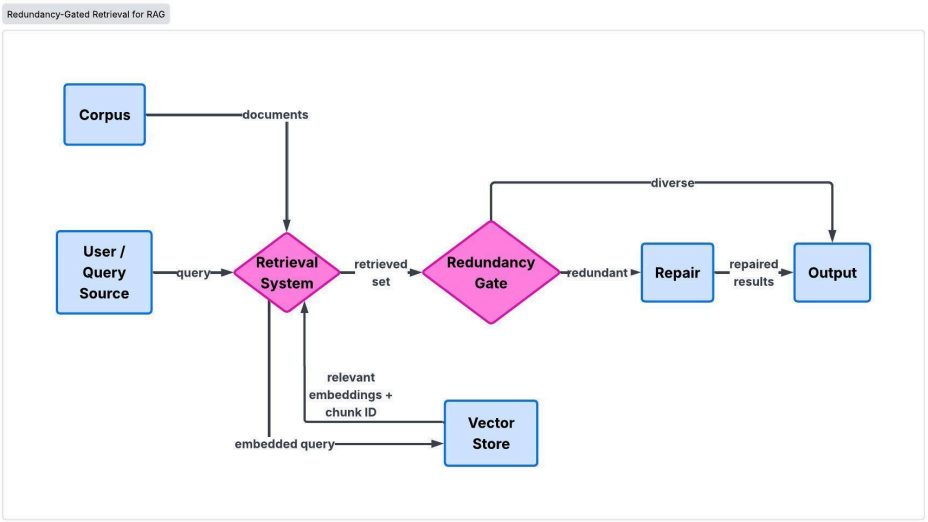


Fig. 4.2. Data flow diagram

Fig. 3. Data flow diagram of the redundancy-gated retrieval pipeline, showing corpus embedding, query retrieval via the vector store, and the redundancy gate routing results to either direct output or repair.

4.2.2 Use Case Diagram

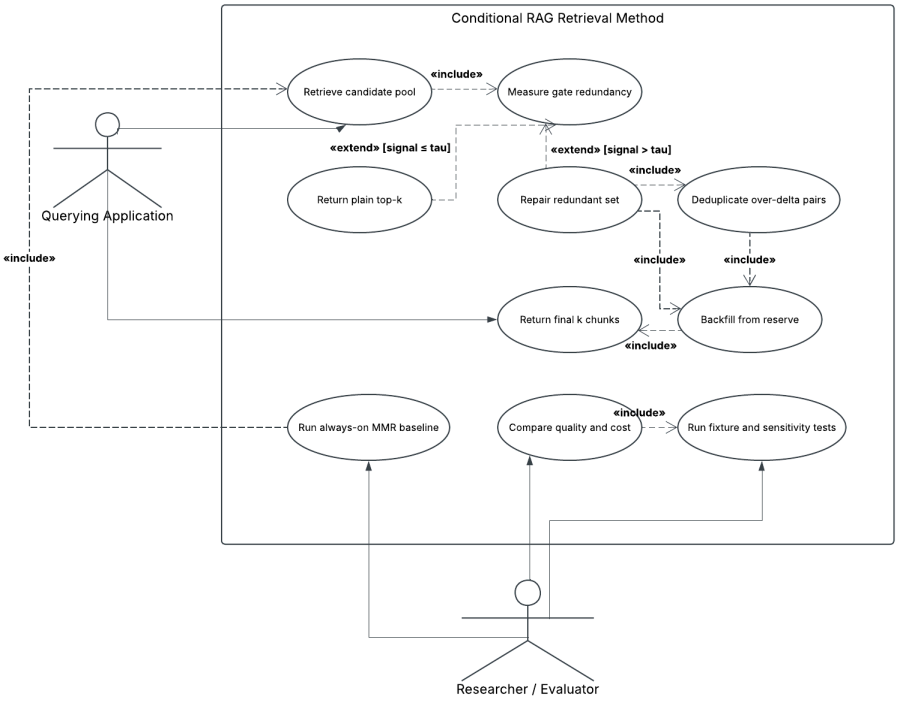


Fig. 4.3. Use case diagram

4.2.3 Class Diagram

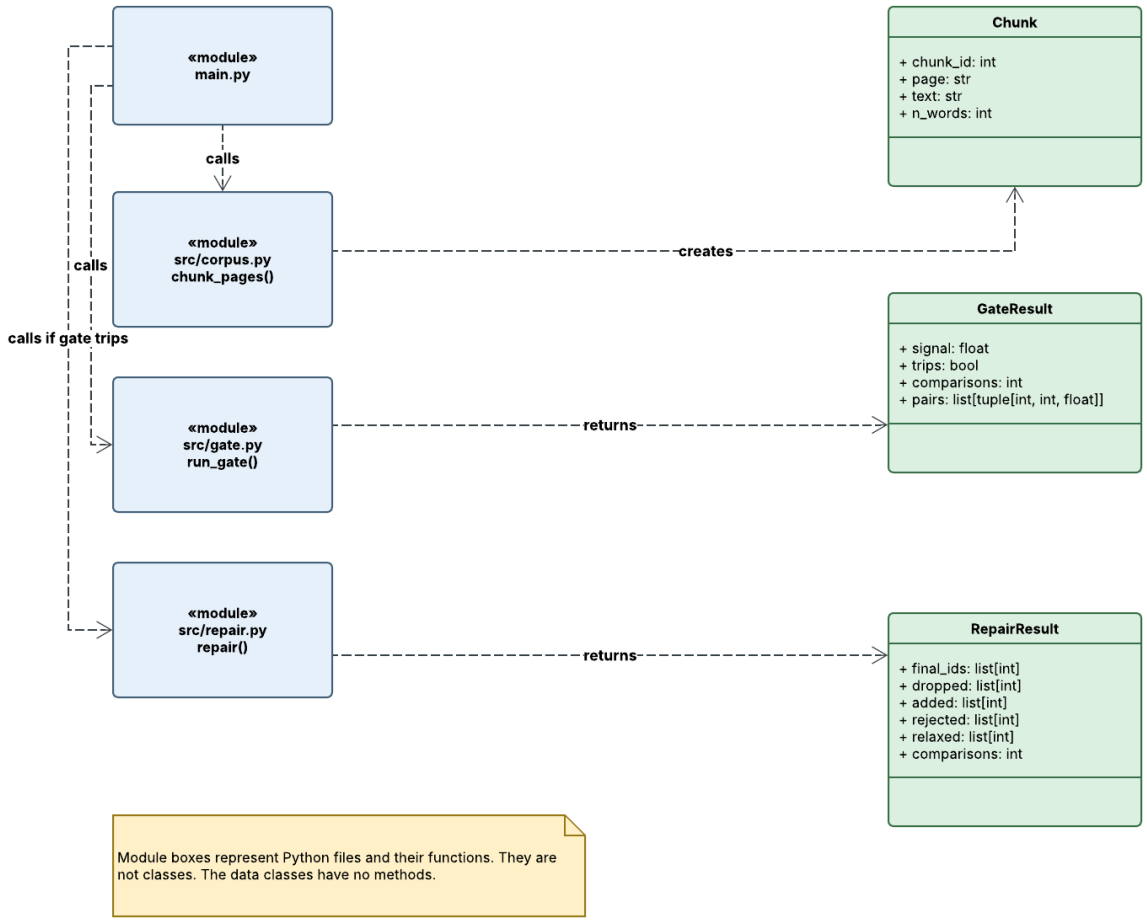


Fig. 4.4. Class Diagram

4.2.4 Sequence diagram

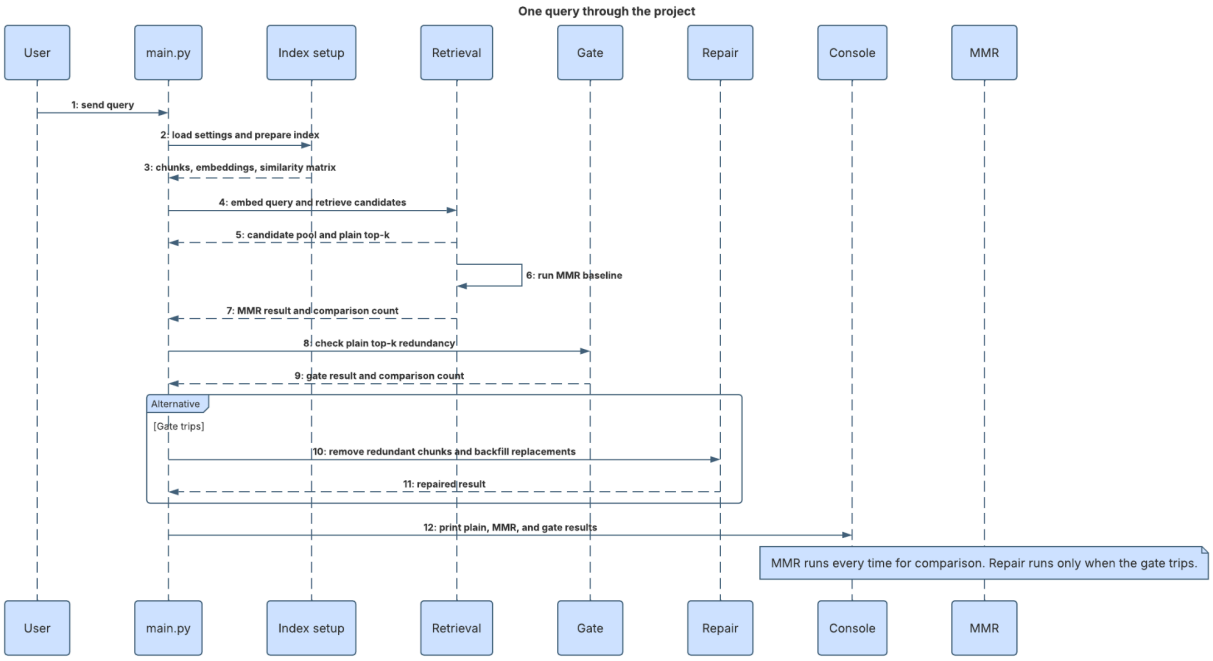


Fig. 4.5. Sequence diagram

CHAPTER 5

5. METHODOLOGY AND TESTING

5.1 Methodology

5.1.1 Pipeline

For each query:

1. **Retrieve a pool.** Fetch the top pool_size chunks by query-to-chunk similarity, where pool_size is comfortably larger than k.
2. **Take the plain top-k.** The k most relevant chunks. The rest of the pool is the backfill reserve.
3. **Gate check.** Compute the mean similarity between the chunks in the top-k. If it is at or below tau, ship the top-k unchanged and stop. This is the common, cheap path.
4. **Repair, only if the gate trips.** Remove redundant chunks, then backfill from the reserve.
5. **Return the final k chunks** to the generation step.

5.1.2 The gate

The signal is the simple mean of the C(k,2) pairwise chunk-to-chunk similarities in the top-k. That is 3 lookups at k=3 and 10 at k=5. If the mean is greater than tau, the gate trips. This cost is fixed whether or not it trips.

Tau is derived from delta and k, so it is correct for any k:

$$\tau = \frac{\delta + (C(k,2)-1)b}{C(k,2)}$$

In words, the gate trips when the set is at least as redundant as one duplicate pair inside an otherwise normal top-k. Here b is the typical similarity of two non-duplicate chunks sharing a top-k. It is measured by treating each of the 136 chunks as a query, taking its top-k from the other 135, and using the median of all pair similarities at or below delta.

Table 5.1 Delta and Tau Values

k	b	delta	tau
3	0.782	0.856	0.807

5	0.772	0.856	0.780

Tau is computed at the start of every run by compute_tau in src/calibrate.py, so it never goes stale.

5.1.3 The repair

Deduplication. Find the single most similar pair in the top-k above delta and drop its lower-relevance member. Recheck the remaining set and repeat until no pair exceeds delta. Dropping one chunk at a time avoids over-pruning. With A-B 0.95, B-C 0.90 and A-C 0.70, the method drops B and keeps {A, C}.

Backfill. Walk the reserve in relevance order. Reject a candidate if its similarity to any kept chunk exceeds delta, otherwise keep it. Stop when the set is back at k or the reserve is exhausted.

If the reserve runs out, take back the rejected candidate least similar to the kept set, and repeat until there are k chunks. This keeps the return size fixed at k, at the cost of the no-duplicates guarantee for that rare query.

5.1.4 Setting delta

Two chunks are duplicates if they share about 75 percent of their content. Delta is calibrated as follows:

1. Split every chunk into sentences.
2. Build one fake duplicate per chunk by keeping 75 percent of its sentences in order and replacing the rest with random sentences from chunks on other pages. Round the sentence count at random so the average share is exactly 75 percent.
3. Score each fake against its original by cosine similarity.
4. Take the 5th percentile of the 136 scores.
5. Repeat over 5 seeds and average.

The result is **delta = 0.856**. Sentences are replaced, not deleted, because a copy with sentences deleted is a strict subset of the original and scores unrealistically high.

5.1.5 MMR baseline

MMR is the always-on baseline for comparison. Starting from an empty set, each round picks the remaining candidate with the highest score, until k are chosen:

$$score = \lambda \cdot rel(q, c) - (1 - \lambda) \cdot max\, sim(c, s)$$

5.2 Testing

5.2.1 Setup

Plain top-k, always-on MMR and the gate method were run on the same 100 queries: 50 narrow (one fact from one chunk) and 50 broad (topics covered by at least two articles). The queries were written from the corpus and reviewed, then frozen before the first run. Settings were k = 3, pool 15, delta = 0.856, tau = 0.806 and MMR lambda = 0.7.

5.2.2 Metrics Used

Table 5.2 Metrics

Metric	What it measures	Better
Trigger rate	Share of queries where the gate trips and repair runs	Lower means more work skipped
Compute cost	Average similarity comparisons per query, counted in code rather than estimated	Lower
ILS	Mean pairwise similarity inside the final set	Lower means less redundancy
Relevance	Mean query-to-chunk similarity of the final set	Higher

Trigger rate and cost are read together, and ILS and relevance are read together. ILS alone can be gamed by returning unrelated chunks. Results are also split into tripped and not-tripped queries, because the gate only changes the tripped ones.

5.2.3 How the tests are run

Building the query set. The 50 narrow queries come from 50 chunks drawn at random (fixed seed 0) from the 136, with one question aimed at a specific fact in each chunk. The 50 broad queries cover topics that at least two of the three articles discuss, such as symptoms, causes,

diagnosis and management. The set is frozen before the first run, and delta and tau come from the corpus alone, never from these queries, so the queries are used only for evaluation.

Running a query. All three methods work in the same embedding space. Embeddings are normalized, so similarity is a dot product, and the full chunk-to-chunk similarity matrix is computed once and cached. For each query:

1. Retrieve a pool of 15 chunks by query-to-chunk similarity.
2. **Plain** returns the top 3 of the pool.
3. **MMR** picks 3 chunks round by round from the pool using the lambda score.
4. **Gate** takes the same top 3, checks the gate, and repairs only if it trips.

Counting cost. Each lookup in the cached matrix counts as one comparison, and the code counts them as they happen rather than estimating from a formula. At k = 3 the gate costs 3 comparisons plus whatever the repair spends on queries that trip. MMR costs a flat 40 on every query. The lookups used to calibrate delta and tau are not counted, since calibration is not part of answering a query.

Recording results. For every query and method the harness records the final set, the number of comparisons, ILS, relevance, whether the gate tripped, and the gate signal. These rows go to a per-query file for spot-checking. The numbers are then averaged over all 100 queries, over tripped and not-tripped queries separately, and over broad and narrow queries separately.

Repeating across k. The whole run is repeated at k = 3, 5, 10 and 15. Tau is re-derived from delta and k for each value, so trigger rate is a measured result rather than a setting.

CHAPTER 7

7. RESULTS AND DISCUSSION

7.1 Results

All 100 queries were run with k = 3 (3 chunks returned), a pool of 15 candidates, delta = 0.856, tau = 0.806 and MMR lambda = 0.7. Delta is the line for calling two chunks duplicates, and tau is the line for calling a whole set too redundant. Tau is calculated automatically, so the earlier sections show it rounded to 0.807.

7.1.1 Headline

On average the gate method spends 4.26 similarity comparisons per query, against 40 for always-on MMR, which is about 11 percent of MMR's cost. It repairs 24 percent of queries. On those 24, ILS falls from 0.825 to 0.795 while relevance only moves from 0.729 to 0.724. On the other 76, the gate returns the plain top-k unchanged, while MMR still changed 47 of them and paid the full 40 comparisons each time.

In simple terms, the gate does about one ninth of MMR's work, fixes the queries that need it, and leaves the rest alone.

7.1.2 Metrics 1 and 2: trigger rate and compute cost

A "comparison" is one similarity lookup, and the trigger rate is how often the gate decides a repair is needed.

Table 7.1 Metrics 1,2 comparison

Method	Avg comparisons per query	Trigger rate
Plain top-k	0	n/a
MMR	40.0	n/a (runs on every query)
Gate	4.26	24% (24 of 100)

The gate's 4.26 is an average of two groups. The 76 queries that did not trip cost 3 comparisons each, just the gate check. The 24 that did trip cost 8.25 on average, which is 3 for the gate plus 5.25 for the repair.

The gap widens as k grows. Tau is re-derived for every k.

Table 7.2 MMR vs Gate Trip comparison

k	tau	MMR	Gate	Gate as share of MMR	Trigger rate
3	0.806	40	4.26	11%	24%
5	0.780	220	18.41	8%	46%
10	0.760	1,965	91.42	5%	44%
15	0.749	6,860	224.58	3%	43%

MMR's work explodes as it returns more chunks (6,860 comparisons at k = 15), while the gate stays small, so its share of MMR's cost falls from 11 percent to 3 percent.

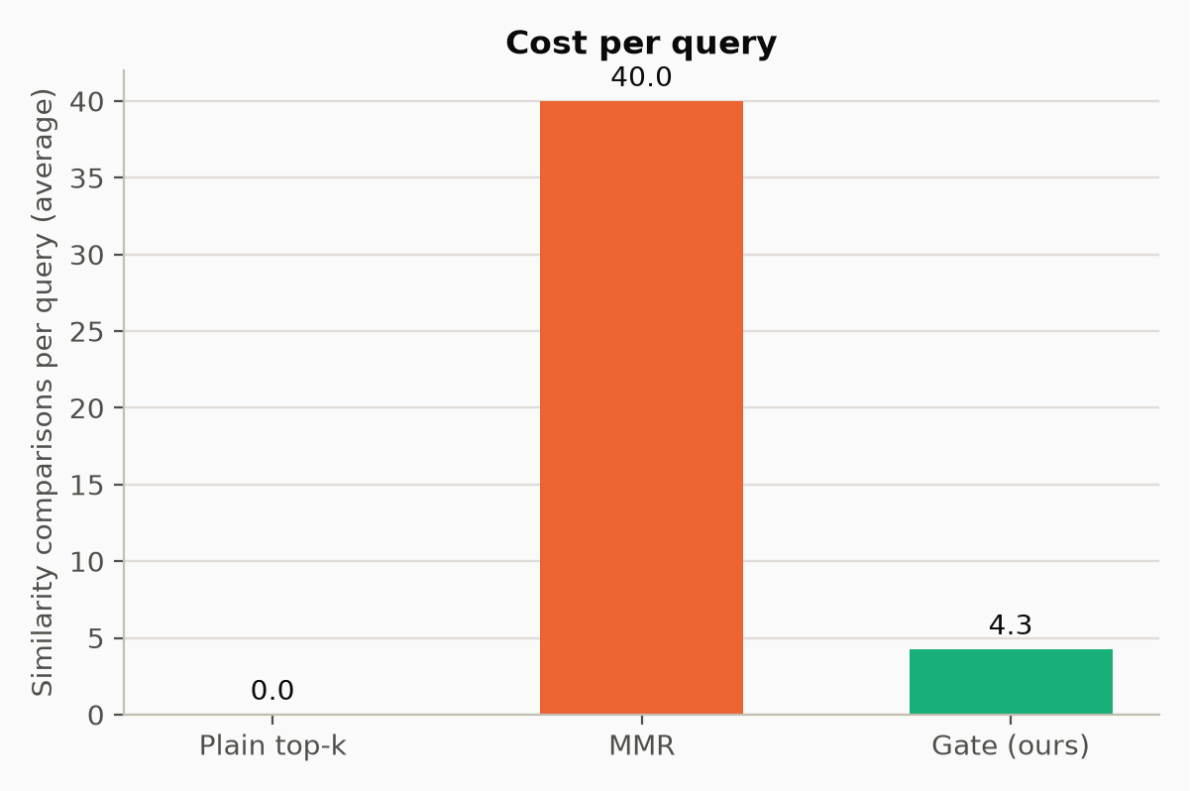


Fig 7.1 Cost per query comparison

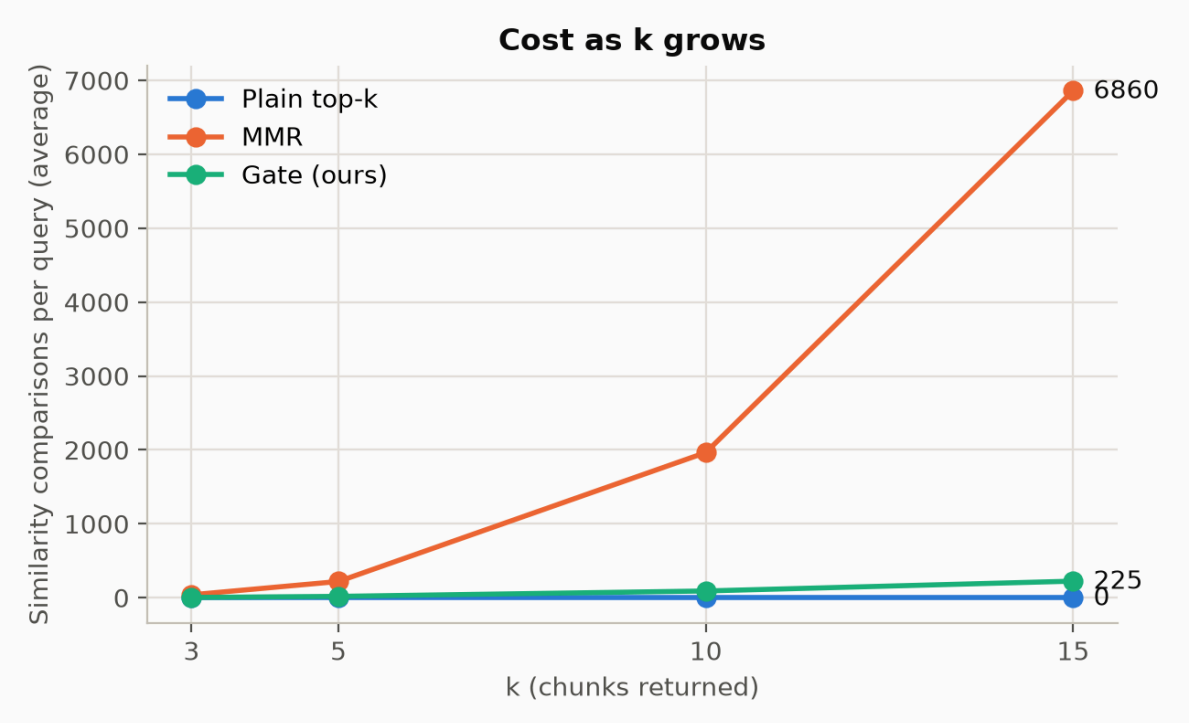


Fig 7.2 Cost as k grows comparison

7.1.3 Metrics 3 and 4: ILS and relevance

ILS measures how similar the returned chunks are to each other, so lower means more variety. Relevance measures how well the chunks match the query, so higher is better. They are read together, because variety is only useful if relevance holds.

Table 7.3 Metrics 3, 4 comparison

Queries	Method	ILS (lower is better)	Relevance	Changed vs plain
All 100	Plain	0.786	0.714	0%
	MMR	0.752	0.709	65%
	Gate	0.778	0.713	15%
Tripped (24)	Plain	0.825	0.729	0%
	MMR	0.774	0.722	75%
	Gate	0.795	0.724	63%
Not tripped (76)	Plain	0.773	0.709	0%
	MMR	0.745	0.705	62%
	Gate	0.773	0.709	0%

Table 7.3 Metrics 3, 4 comparison

What the table shows:

- Where the gate trips, it lands between plain and MMR. It removes some redundancy

(ILS 0.825 to 0.795) and costs almost no relevance.

- Where it does not trip, it is identical to plain. MMR still changes 62 percent of these queries, for a small variety gain (ILS 0.773 to 0.745) and a small relevance loss, at 40 comparisons each.
- Overall, the gate keeps relevance almost exactly as it was (0.714 to 0.713), while MMR gives up a little more (0.709).

Broad and narrow queries behave alike: the trigger rate is 26 percent on broad and 22 percent on narrow, and the three methods rank the same way in both groups. The query mix is therefore not what drives the result.

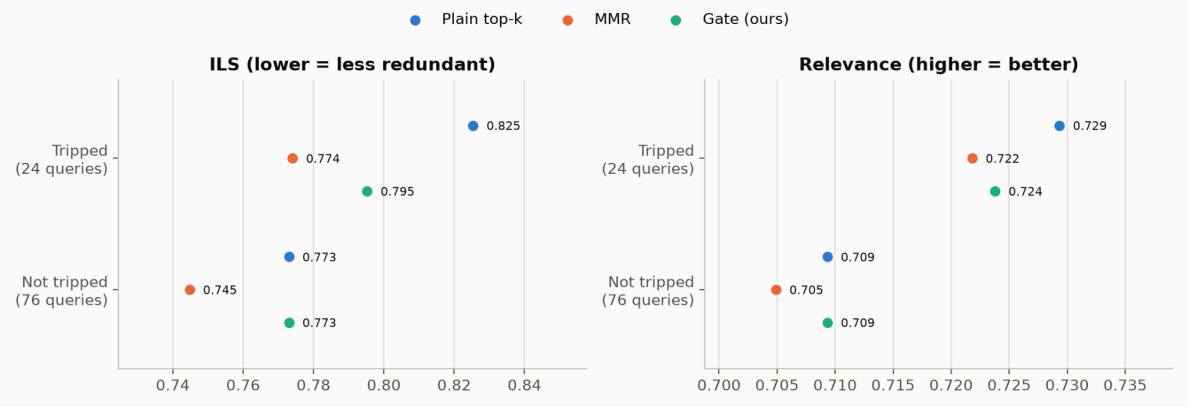


Fig 7.3 ILS, Relevance comparison

CHAPTER 8

8. CONCLUSION

This project developed a redundancy-gated retrieval approach for reducing repeated information in RAG context. Unlike always-on diversification methods such as MMR, the approach first checks the plain top-k results and runs repair only when their average pairwise similarity exceeds a threshold. Repair removes the less relevant chunk from redundant pairs and backfills the available slots with less similar candidates.

The prototype was evaluated on 100 queries using the three-page diabetes corpus. The gate triggered on 24% of queries and averaged 4.26 similarity comparisons per query, compared with 40 for MMR. Its average intra-list similarity was 0.7783, slightly lower than plain top-k at 0.7856, while its average relevance was 0.7128 compared with 0.7141 for plain top-k. MMR achieved lower intra-list similarity at 0.7518, with average relevance of 0.7090. On this corpus, the gate therefore reduced comparison cost substantially while making a modest reduction in redundancy and largely preserving relevance.

Overall, this project presents a simple, measurable alternative to applying diversification on every query. Further evaluation on a larger and more varied corpus would help determine how well the method generalizes and how its cost and relevance compare with MMR in practice.

CHAPTER 9

9. REFERENCES

1. Carbonell, J. and Goldstein, J. "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries." In Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '98), pp. 335–336. ACM, 1998.

2. Vieira, M.R., Razente, H.L., Barioni, M.C.N., Hadjieleftheriou, M., Srivastava, D., Traina, A., and Tsotras, V.J. "On Query Result Diversification." In 2011 IEEE 27th International Conference on Data Engineering (ICDE), pp. 1163–1174. IEEE, 2011.

3. Drosou, M. and Pitoura, E. "Search Result Diversification." ACM SIGMOD Record, 39(1), pp. 41–47, 2010.

4. Drosou, M. and Pitoura, E. "Diverse Set Selection Over Dynamic Data." IEEE Transactions on Knowledge and Data Engineering, 26(5), pp. 1102–1116, 2014.

5. Zheng, K., Wang, H., Qi, Z., Li, J., and Gao, H. "A Survey of Query Result Diversification." Knowledge and Information Systems, 51(1), pp. 1–36, 2017.

6. Y. Huang and J. X. Huang, "A survey on retrieval-augmented text generation for large language models," ACM Computing Surveys, vol. 58, no. 12, Art. no. 300, May 2026.

7. Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., Tonellotto, N., and Silvestri, F. "The Power of Noise: Redefining Retrieval for RAG Systems." In Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24), pp. 719–729. ACM, 2024.

8. Salemi, A. and Zamani, H. "Evaluating Retrieval Quality in Retrieval-Augmented Generation." In Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24), pp. 2395–2400. ACM, 2024.

9. Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.S., and Li, Q. "A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models." In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24), pp. 6491–6501. ACM, 2024.

10. Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., and Liu, T. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions." ACM Transactions on Information Systems, 43(2), pp. 1–55, January 2025.

11. Agarwal, S., Sundaresan, S.N., Mitra, S., Mahapatra, D., Yu, T., and Saini, S. "Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation." In Proceedings of the 2025 ACM SIGMOD International Conference on Management of Data (SIGMOD '25). ACM, 2025.

APPENDIX A – Sample Code

```
FUNCTION CheckGate(top_k, S, tau)

   IF length(top_k) < 2:
      REPORT an error because at least two chunks are needed

   total_similarity ← 0
   pair_count ← 0

   FOR each unique pair (i, j) in top_k:
      total_similarity ← total_similarity + S[i][j]
      pair_count ← pair_count + 1

   signal ← total_similarity / pair_count

   IF signal > tau:
      RETURN signal, "TRIPPED"
   ELSE:
      RETURN signal, "NOT TRIPPED"

END FUNCTION
```